My Title

Duc Minh Le[1] and Author 2[2]

[1] Department of Information Technology,
Swinburne Vietnam, FPT University
duclm20@fe.edu.vn
[2] Organisation 2, Hanoi, Vietnam
author2@org2.vn

Abstract. Domain-driven design (DDD) has widely been used to develop RESTful software in a range of programming language platforms. The use of code generators in these technologies helps significantly increase productivity and achieve large scale software reuse. However, there been no work that address incremental construction of RESTful full stack software (RFS) at both the module and software levels. In this paper, we propose a generative module-based method for RFS to bridge this gap. We characterise RFS and realise this in a module-based DDD software architecture, named MOSA^R. Our method takes as input a software configuration and the required software assets and automatically generates a module-based RFS software. We present algorithms for the frontend and backend generation functions. The backend software consist of web service modules. The frontend software consists of modules that are designed with single-page views. Each view can be nested to reflect the containment tree of the module. We demonstrate by implementing the generators for two popular platforms (React and Spring Boot). The evaluation shows that the generators are able to support large software.

Keywords: kw1, kw2.

1 Introduction

The rest of the paper is structured as follows. Section...

2 Background

In this section, we present a motivating software example named CourseMan and a number of key concepts of the paper.

Example: CourseMan. To illustrate the concepts presented in this paper, we adopt the software example, named course management (CourseMan), that we used in previous works [2,1]. This example is scoped around a domain model whose elements are represented by the following fundamental UML [3] meta-concepts: class, attribute, operation, association, association class and generalisation. The CourseMan's domain model consists of six domain classes and associations between them. Class `Student` represents the domain concept Student Class `CourseModule` represents the `CourseModules` that are offered to students. Class `Address` represents the addresses where `Students` live as they undertake their studies. The study arrangement involves enrolling each `Student` into one or more `CourseModules`. Class `Enrolment` represents the enrollment records.

```
@RFSGenDesc(
  stackSpec = StackSpec.FS,
  execSpec = ExecSpec.Gen,
  genMode = GenerationMode.SOURCE_CODE,
  feProjPath = "/home/ducmle/tmp/restfstool-fe",
  feProjName = "fe-courseman",
  feProjResource = "src/main/resources/vuejs",
  fePlatform = FEPlatform.VUE_JS,
  feOutputPath = "/home/projects/jda//CourseManVueJs_Gen",
  feServerPort = 5000,  // default: 3000
  feAppClass = FEApp.class,
  feThreaded = true,
  beLangPlatform = LangPlatform.SPRING,
  bePackage = "org.jda.example.coursemanrestful.modules",
  beOutputPath = "/home/projects/jda/examples/courseman/mosar/src/main/java",
  beTargetPackage = "org.jda.example.coursemanrestful.backend"
  , beAppClass = BESpringApp.class,
  beServerPort = 8080  // default: 8080
)
@SystemDesc(
  appName = "Courseman",
  splashScreenLogo = "coursemanapplogo.jpg",
  modules = {
    ModuleMain.class,
    ModuleCourseModule.class,
    ModuleEnrolment.class,
    ModuleStudent.class,
    ModuleAddress.class,
    ModuleStudentClass.class
  }
public class SCCCourseMan { }
```

```
@@RFSGenDesc((
  stackSpec = StackSpec.FE,
  execSpec = ExecSpec.Gen,
  genMode = GenerationMode.SOURCE_CODE,
  feProjPath = "/home/ducmle/tmp/restfstool-fe",
  feProjName = "fe-courseman",
  feProjResource = "src/main/resources/angular",
  fePlatform = FEPlatform.VUE_JS,
  feOutputPath = "/home/projects/jda//CourseManVueJs_Gen",
```

```
  feServerPort = 5000,  // default: 3000
  feAppClass = FEApp.class,
  feThreaded = true,
  beLangPlatform = LangPlatform.SPRING,
  bePackage = "org.jda.example.coursemanrestful.modules",
  beOutputPath = "/home/projects/jda/examples/courseman/
      mosar/src/main/java",
  beTargetPackage = "org.jda.example.coursemanrestful.
      backend"
  , beAppClass = BESpringApp.class,
  beServerPort = 8080  // default: 8080
)
@SystemDesc(
  appName = "Courseman",
  splashScreenLogo = "coursemanapplogo.jpg",
  language = Language.English,
  orgDesc = @OrgDesc(name = "Faculty of IT",
          address = "Hanoi, Vietnam",
          logo = "hanu.gif",
          url = "http://swinburne.edu.vn"),
  dsDesc = @DSDesc(
          type = "derby",
          dsUrl = "data/domainds",
          user = "admin",
          password = "password",
          dsmType = DSM.class,
          domType = DOM.class,
          osmType = JavaDbOSM.class,
          connType = ConnectionType.Embedded),
  modules = {
          ModuleMain.class,
          ModuleCourseModule.class,
          ModuleEnrolment.class,
          ModuleStudent.class,
          ModuleAddress.class,
          ModuleStudentClass.class
  },
  sysModules = {},
  setUpDesc = @SysSetUpDesc(setUpConfigType = SetUpConfig.
      class),
  securityDesc = @SecurityDesc(isEnabled = false))
public class SCCCourseMan { }
```

Concept 1.

For example, ...

Concept 2.

For example, ...

3 Content 1

4 Content 2

5 Tool Support and Evaluation

We implemented our method ...

Evaluation 1. We analyse the time complexities of two generator functions (`FEGen` and `BEGen`) to show that they run in **quadractic time**. This is scalable to support software with large domain models. Due to space restriction, we do not present the proofs in this paper. Refer to our extended paper for details.

Discussion.

6 Related Work

Our work in this paper is positioned at the intersections among three areas: ...

Area 1

Area 2

7 Conclusion

In this paper, we proposed a solution to address the problem of ...

Our method is novel in ...

We contend that our contributions in this paper constitute a novel contribution to ....

In future work, we plan to ...

References

1. Le, D.M., Dang, D.H., Nguyen, V.H.: Generative Software Module Development for Domain-Driven Design with Annotation-Based Domain Specific Language. Information and Software Technology **120**, 106–239 (Apr 2020)
2. Le, D.M., Dang, D.H., Vu, H.T.: jDomainApp: A Module-Based Domain-Driven Software Framework. In: Proc. 10th Int. Symp. on Information and Communication Technology (SOICT). pp. 399–406. ACM, New York, USA (2019)
3. OMG: Unified Modeling Language version 2.5 (2015), http://www.omg.org/spec/UML/2.5/